

# YapNow

## Domain-Driven Nano-Services

Coding & Architecture Guidelines

A simple structure for building, reviewing, and scaling YapNow backend services.

### CORE IDEA

Business ownership should be visible from the folder, file, and function name.

## 1. Why Nano-Services for YapNow?

YapNow has clear business concepts such as Customer Profile, Matchmaking, Conversation, Payment & Subscription, Notification, and Customer Support. The nano-service approach keeps each business concept small, focused, and easy to find.

| Traditional / Large Service                        | YapNow Nano-Service                                    |
|----------------------------------------------------|--------------------------------------------------------|
| Large service grows around technical modules       | Service grows around one business concept              |
| Logic can spread across many folders               | Logic stays inside its owning domain                   |
| Shared utils/helpers become common dumping grounds | Shared logic is avoided; ownership stays clear         |
| A change may touch unrelated areas                 | A change is usually contained within the owning domain |
| New developer must learn internal conventions      | Folder + file names explain ownership                  |

The goal is not “more services”. The goal is **clear business boundaries**.

## 2. YapNow Service Landscape

| Customer Profile                            | Matchmaking                                   | Conversation                           | Payment & Subscription      |
|---------------------------------------------|-----------------------------------------------|----------------------------------------|-----------------------------|
| User identity<br>preferences<br>permissions | Request → pool<br>filters → score<br>match_id | conversation_id<br>chat / call / video | payments<br>plans<br>wallet |

## 3. Inside One Nano-Service

```
src/  
  ■■■ clients/                # other services  
  ■   ■■■ customer_profile_service/  
  ■   ■■■ payment_subscription_service/  
  ■  
  ■■■ service/  
    ■■■ matchmaking/  
      ■■■ index.ts           # L0 – API/router  
      ■■■ application/       # L1 – use-case flow  
      ■   ■■■ connect_user.ts  
      ■   ■■■ events/  
      ■     ■■■ handle_connection_ended.ts
```

```

■■■ domain/
■■■ types.ts
■■■ request/
■■■ pool/
■■■ eligibility/
■■■ scoring/
■■■ match/

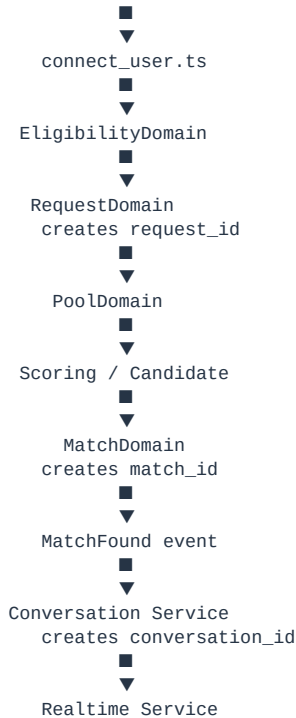
```

# L2 – business

|    |             |                            |                                     |
|----|-------------|----------------------------|-------------------------------------|
| L0 | Router      | Routes only                | No business logic                   |
| L1 | Application | Sequences the use case     | No DB / no business decisions       |
| L2 | Domain      | Owns business rules + data | No HTTP / no sibling-domain imports |

## 4. Real YapNow Example — User Connects

User selects CHAT / VOICE / VIDEO



## 5. Three Rules to Remember

| 01<br>SEALED DOMAINS                                                           | 02<br>FLOW vs DECISION                                | 03<br>NAMES EXPLAIN                          |
|--------------------------------------------------------------------------------|-------------------------------------------------------|----------------------------------------------|
| Matchmaking scoring does not import eligibility. Pool does not import scoring. | Application sequences. Domain decides business rules. | Prefer calculateMatchScore over processData. |

## 6. Matching Example

The user chooses preferences before connecting. Store them as a **preference snapshot** for that request.

```

{
  "communicationType": "VOICE",
  "location": { "type": "CITY", "value": "Bangalore" },
  "language": ["English", "Hindi"],
  "gender": "ANY",
  "profession": ["Software Engineer"],
  "passion": ["Technology", "Travel"]
}

```

**Example score:** Language 30 + Location 20 + Gender 15 + Passion 20 + Profession 10 = **95**.

## 7. Simple Application Code

```
// application/connect_user.ts

const connectUser = async (
  input: ConnectUserInput,
): Promise<{ requestId: string }> => {
  const { userId, communicationType, preferences } = input;

  await EligibilityDomain.isUserEligible(
    userId,
    communicationType,
  );

  const requestId = await RequestDomain.createRequest(
    userId,
    communicationType,
    preferences,
  );

  await PoolDomain.addRequestToPool(
    requestId,
    userId,
    communicationType,
    preferences,
  );

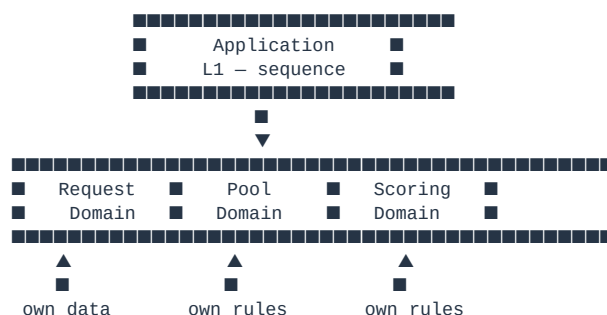
  return { requestId };
};
```

Notice: no database query, no match-score calculation, and no direct call to another business service.

## 8. Good vs Bad

| BAD                       | GOOD                                     |
|---------------------------|------------------------------------------|
| processData()             | calculateMatchScore()                    |
| checkUser()               | isUserEligible()                         |
| MatchManager              | match/ domain                            |
| application → MatchModel  | application → MatchDomain                |
| application → Payment API | domain/client boundary → Payment Service |
| utils/matching.ts         | scoring/calculate_match_score.ts         |

## 9. Ownership & Direction



Never:

Request Domain ■X→ Pool Domain

Pool Domain ■X→ Scoring Domain

Application ■X→ database

**Hoist up, never sideways.** Shared types/constants can move upward. Business logic should retain a clear owner.

## 10. Why This Is Better for YapNow

| Traditional growth problem                             | Nano-service benefit                                     |
|--------------------------------------------------------|----------------------------------------------------------|
| Matchmaking logic spreads into controllers/helpers     | Matchmaking domains show exactly where each rule belongs |
| Profile, payment, and matching concerns become coupled | Business services remain independent                     |
| Changing scoring risks unrelated code                  | Scoring is isolated inside its domain                    |
| Hard to onboard developers                             | Folder + filename + function name explain ownership      |
| Large shared utility layer grows continuously          | No vague shared/utills dumping ground                    |
| Testing requires the whole service                     | Domain rules can be tested independently                 |

## 11. Naming Standard

```
File:      calculate_match_score.ts
Function:  calculateMatchScore
Type:      MatchRequest
Constant:  MAX_MATCHING_TIME_MS
Domain:    scoring
```

```
Good:
isUserEligible
addRequestToPool
calculateMatchScore
createMatch
releaseReservation
```

```
Avoid:
processData
handleRequest
checkUser
MatchManager
utils
common
shared
```

The filename and function name should describe the same action.

## 12. PR Review — Quick Checklist

- One nano-service = one business concept.
- Only application/ and domain/ inside the nano-service.
- No utils/common/shared/helpers/manager folders.
- No sibling-domain imports.
- Application does not access the database.
- Application does not call external business services directly.
- One API = one application file.
- Domain index.ts exports only public business functions.
- Names describe business intent.
- No any or silent catch.
- One table = one owning domain.
- Matchmaking creates match\_id; Conversation creates conversation\_id.

## 13. The YapNow Rule of Thumb

### ASK ONE QUESTION:

“Which business concept owns this rule?”

Put the rule there. Keep the application as the flow. Keep domains sealed. Name files and functions so the code explains itself.